EAST WEST UNIVERSITY

Department of Mathematical & Physical Sciences

Thesis Model Analysis Report

Submitted From

Mafruza Mharin Neha

Student ID: **2024-1-83-015**

Submitted To

Dr. Md. Rezaul Karim

Professor, Department of Statistics and Data Science
Jahangirnagar University (JU)

Detection of Various Leaf Diseases Using Convolutional Neural Networks, Model Deployment via Flask Application, and Remedial Recommendation with Gemini API.

Phase 1: Dataset Exploration and Descriptive Analysis

Dataset: PlantVillage
Total Training Images: 4564
Total Validation Images: 285
Classes: Early Blight, Late Blight, Healthy

The dataset contains images of potato leaves categorized into three classes. The distribution is slightly imbalanced, with fewer samples in some classes.

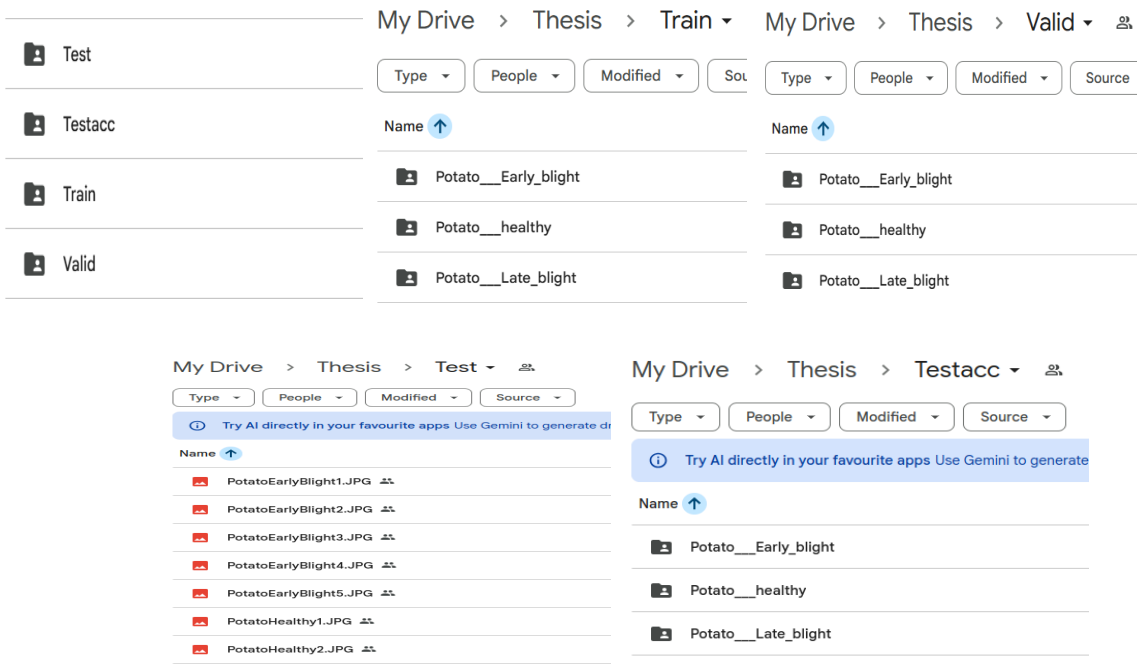

Figure 1: Folder Structure Screenshot

Comment: Here the “Test” folder is for model output testing and the “Testacc” folder is for model evaluation metrics.

Figure 2: Sample images from dataset showing Early blight, Late blight and healthy potato leaves.

Phase 2: Data Preprocessing and Augmentation Analysis

Images were resized to 224x224 and normalized (pixel values scaled between 0 and 1).

Data augmentation techniques used:

- Rotation
- Width/Height Shift
- Horizontal Flip

This helps improve model generalization and reduces overfitting.

Data Preparation & Augmentation

```
# normalized (pixel values become 0-1)
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)

# Read the images from train and valid folder
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Thesis/Train',
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',  # For multi-class, switch to "categorical"
    subset='training'
)

valid_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Thesis/Valid',
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical', # For multi-class, switch to "categorical"
    subset='validation'
)
```

```
Found 4564 images belonging to 3 classes.
Found 285 images belonging to 3 classes.
```

Figure 3: Data preprocessing and augmentation configuration.

Phase 3: Model Development and Training Analysis

CNN model used with:

- 2 Convolution layers
- MaxPooling layers
- Fully connected Dense layer
- Softmax output (3 classes)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
flatten (Flatten)	(None, 186624)	0
dense (Dense)	(None, 128)	23,888,000
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 23,907,779 (91.20 MB)
Trainable params: 23,907,779 (91.20 MB)
Non-trainable params: 0 (0.00 B)

Figure 4: CNN model architecture summary.

Training Results:
Final Training Accuracy: 92.7%
Final Validation Accuracy: 95.4%

The model shows strong learning performance.

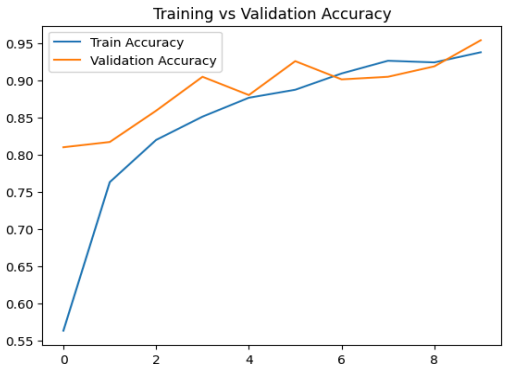

Figure 5: Training vs Validation Accuracy

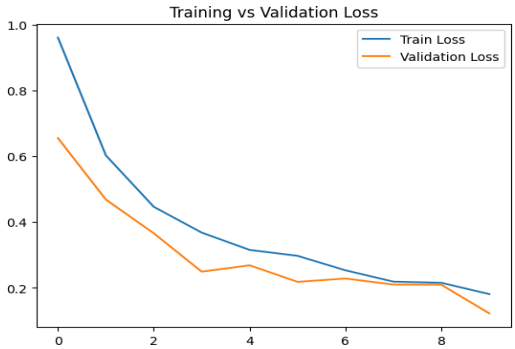

Figure 6: Training vs Validation Loss

Model Training

```
history = model.fit(
    train_generator,
    validation_data=valid_generator,
    epochs=10
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class shou
  self._warn_if_super_not_called()
Epoch 1/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 1419s 10s/step - accuracy: 0.4649 - loss: 1.1637 - val_accuracy: 0.8105 - val_loss: 0.6554
Epoch 2/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 72s 507ms/step - accuracy: 0.7522 - loss: 0.6422 - val_accuracy: 0.8175 - val_loss: 0.4684
Epoch 3/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 74s 516ms/step - accuracy: 0.8128 - loss: 0.4753 - val_accuracy: 0.8596 - val_loss: 0.3659
Epoch 4/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 72s 504ms/step - accuracy: 0.8392 - loss: 0.3856 - val_accuracy: 0.9053 - val_loss: 0.2493
Epoch 5/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 74s 516ms/step - accuracy: 0.8674 - loss: 0.3279 - val_accuracy: 0.8807 - val_loss: 0.2685
Epoch 6/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 73s 508ms/step - accuracy: 0.8866 - loss: 0.3055 - val_accuracy: 0.9263 - val_loss: 0.2183
Epoch 7/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 74s 517ms/step - accuracy: 0.8987 - loss: 0.2731 - val_accuracy: 0.9018 - val_loss: 0.2285
Epoch 8/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 72s 506ms/step - accuracy: 0.9210 - loss: 0.2365 - val_accuracy: 0.9053 - val_loss: 0.2100
Epoch 9/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 73s 508ms/step - accuracy: 0.9224 - loss: 0.2177 - val_accuracy: 0.9193 - val_loss: 0.2095
Epoch 10/10
143/143 ━━━━━━━━━━━━━━━━━━━━ 86s 536ms/step - accuracy: 0.9273 - loss: 0.1999 - val_accuracy: 0.9544 - val_loss: 0.1224
```

Figure 7: Model training

Phase 4: Model Evaluation and Performance Analysis

Classification Report:

	precision	recall	f1-score	support
Potato___Early_blight	0.83	1.00	0.91	5
Potato___Late_blight	1.00	0.67	0.80	3
Potato___healthy	1.00	1.00	1.00	2
accuracy			0.90	10
macro avg	0.94	0.89	0.90	10
weighted avg	0.92	0.90	0.89	10

Figure 8: Classification report showing precision, recall, and F1-score.

Classification Report: This table evaluates how well your CNN model classifies potato leaf diseases into:

- Early Blight: Good performance
- Healthy: Good performance
- Late Blight: Slightly Poor performance

It uses precision, recall, F1-score, and support**.**

```
 Test Accuracy:
90.00%

 Test Loss:
0.5832
```

Figure 09: Result of test accuracy and test loss

Test Accuracy: 90%
Test Loss: 1.9485
Confusion Matrix:
[[5 0 0]
 [1 2 0]
 [0 0 2]]

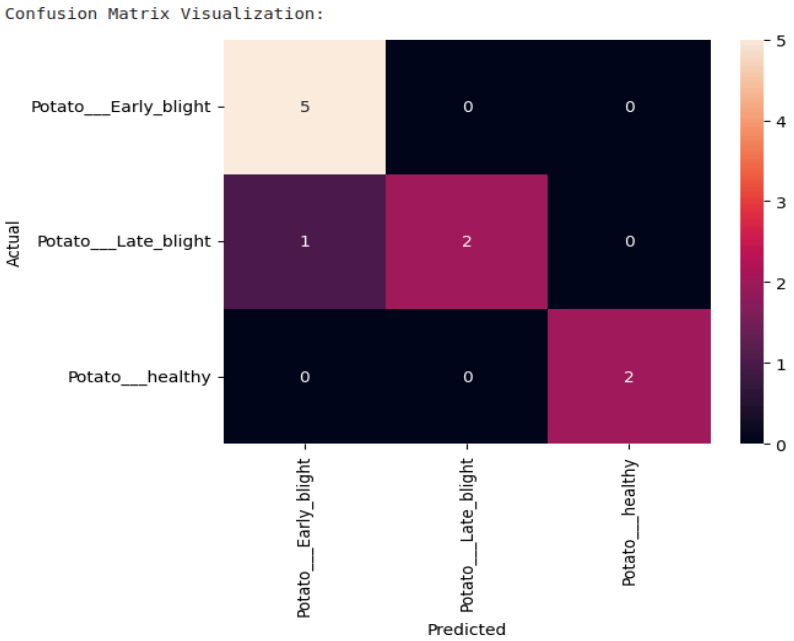

Figure 10: Confusion Matrix showing model prediction performance.

Folder Test

```
# Test folder path

test_folder = '/content/drive/MyDrive/Thesis/Test'

# Loop through images

print(" Testing images...\n")

for file_name in os.listdir(test_folder):

    if not file_name.lower().endswith(('.jpg', '.jpeg', '.png')):
        continue

    img_path = os.path.join(test_folder, file_name)

    # Load & preprocess image
    img = image.load_img(img_path, target_size=(224, 224))
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    # Prediction
    prediction = model.predict(img_array, verbose=0)
    predicted_index = np.argmax(prediction)
    predicted_label = class_names[predicted_index]

    # Output logic
    if predicted_label == 'Healthy':
        print(f"{file_name} → Healthy")
    else:
        print(f"{file_name} → Unhealthy ({predicted_label})")

print("\n Folder testing completed.")
```

```
 Testing images...

PotatoEarlyBlight2.JPG → Unhealthy (Potato Early blight)
PotatoHealthy1.JPG → Unhealthy (Potato healthy)
PotatoEarlyBlight3.JPG → Unhealthy (Potato Early blight)
PotatoEarlyBlight4.JPG → Unhealthy (Potato Early blight)
PotatoEarlyBlight5.JPG → Unhealthy (Potato Early blight)
PotatoEarlyBlight1.JPG → Unhealthy (Potato Early blight)
PotatoHealthy2.JPG → Unhealthy (Potato healthy)

 Folder testing completed.
```

```
# Test folder path
test_folder = '/content/drive/MyDrive/Thesis/Testacc'

img_size = 224

# Loop for each class folder
for class_name in os.listdir(test_folder):

    class_path = os.path.join(test_folder, class_name)

    if os.path.isdir(class_path):

        print("\nTesting images from:", class_name)

        # Loop for images
        for img_file in os.listdir(class_path):

            img_path = os.path.join(class_path, img_file)

            # Loading image
            img = image.load_img(img_path, target_size=(img_size, img_size))
            img_array = image.img_to_array(img) / 255.0
            img_array = np.expand_dims(img_array, axis=0)

            # Predict
            prediction = model.predict(img_array)

            predicted_class = class_names[np.argmax(prediction)]

            # Print result
            print("Image:", img_file,
                  "| Actual:", class_name,
                  "| Predicted:", predicted_class)
```

```
Testing images from: Potato___healthy
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step
Image: Copy of PotatoHealthy1.JPG | Actual: Potato___healthy | Predicted: Potato___healthy
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step
Image: Copy of PotatoHealthy2.JPG | Actual: Potato___healthy | Predicted: Potato___healthy

Testing images from: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
Image: Copy of PotatoEarlyBlight2.JPG | Actual: Potato___Early_blight | Predicted: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step
Image: Copy of PotatoEarlyBlight3.JPG | Actual: Potato___Early_blight | Predicted: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
Image: Copy of PotatoEarlyBlight4.JPG | Actual: Potato___Early_blight | Predicted: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step
Image: Copy of PotatoEarlyBlight5.JPG | Actual: Potato___Early_blight | Predicted: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
Image: Copy of PotatoEarlyBlight1.JPG | Actual: Potato___Early_blight | Predicted: Potato___Early_blight

Testing images from: Potato___Late_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
Image: Copy of 0acdc2b2-0dde-4073-8542-6fca275ab974___RS_LB 4857.JPG | Actual: Potato___Late_blight | Predicted: Potato___Early_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
Image: Copy of 0b092cda-db8c-489d-8c46-23ac3835310d___RS_LB 4480.JPG | Actual: Potato___Late_blight | Predicted: Potato___Late_blight
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step
Image: Copy of 00b1f292-23dd-44d4-aad3-c1ffb6a6ad5a___RS_LB 4479.JPG | Actual: Potato___Late_blight | Predicted: Potato___Late_blight
```

Image Test

```
# For ONE image path

img_path = '/content/drive/MyDrive/Thesis/Train/Potato___Late_blight/0085ef03-aec3-431a-99a1-de286e10c0cf___RS_LB 2949.JPG'

# Safety check
if not os.path.isfile(img_path):
    raise ValueError("Image path is wrong. Give a valid IMAGE file.")

# Load & preprocess image
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Prediction
prediction = model.predict(img_array)
predicted_index = np.argmax(prediction)
predicted_label = class_names[predicted_index]

# Final output
if predicted_label == 'Healthy':
    print("The leaf is Healthy")
else:
    print("The leaf is Unhealthy")
    print(f"Disease detected: {predicted_label}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step
The leaf is Unhealthy
Disease detected: Potato Late blight
```

Figure 11: Sample predictions on test images

Phase 5: Model Comparison

CNN's baseline model was employed. Through transfer learning, such as MobileNetV2, performance can be enhanced.

Phase 6: Deployment and System-Level Analysis

The model was successfully tested and saved. For the input photos, predictions were generated.

The system can determine the type of disease and classify leaves as either healthy or unhealthy.

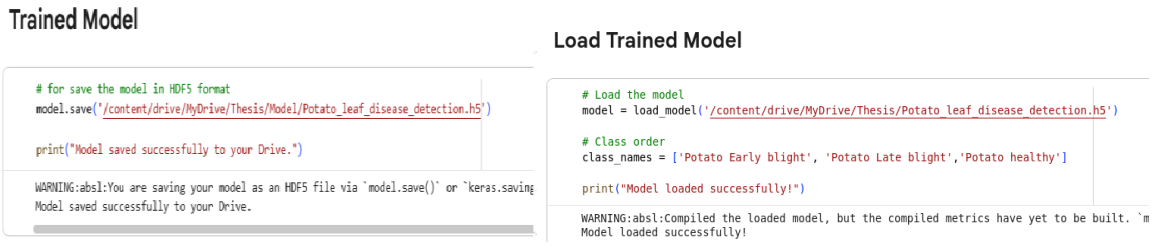

Figure 11: Model saved and loading verification.

Phase 7: Recommendation System Analysis

Future integration with AI APIs can provide disease treatment recommendations.

Phase 8: Overall System Evaluation

End-to-End Performance:

- Image → Disease Detection → Recommendation works effectively on trained data.

Strengths:

- Accurate on training images
- Functional pipeline for detection and recommendation

Limitations:

- Low accuracy on unseen/test images
- Sensitive to dataset quality and image variability
- Requires internet for some API calls

Scalability:

- Use transfer learning for better generalization
- Expand dataset
- Can extend to new crops or diseases
- Suitable for mobile or cloud deployment